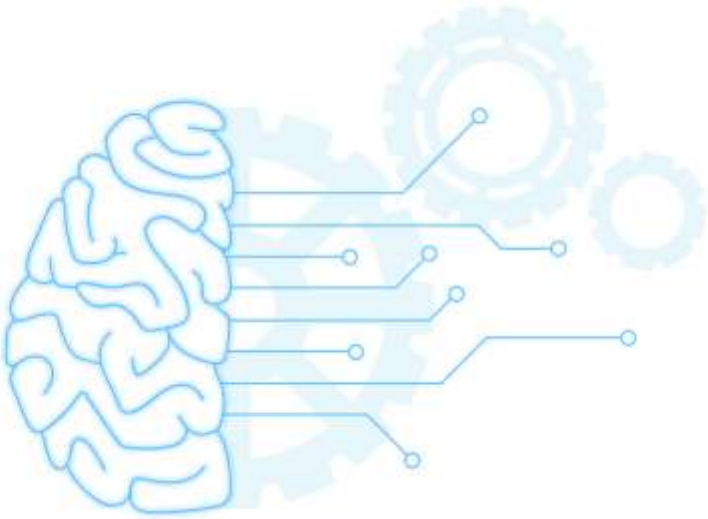
Détection d'Intrusion Réseau par Machine Learning

NIDS-AI



Réalisé par :

- **Abdirachid Barkat Sougueh**
- **Nourdine Abdourahman Moumin**
- **Abdourazak Moussa Said**
- **Raichami Mchami**

Encadré par : Dr Moubarek Barre Hassan

2025/2026

SOMMAIRE

Remerciements 2
INTRODUCTION 3
I. Description du projet 4
A. Fondements du système de detection d'intrusion reseaux 4
B. Objectif Educatif 4
C. Approche Méthodologique 4
II. Collecte des Données 6
A. Source des Données 6
B. Importation des Données 7
C. Prétraitement des Données 7
1. Compréhension des Données 7
2. Nettoyage des Données 8
1. Encodage de données catégorielles en numérique 9
2. Standardisation des Données 9
3. Visualisation des Données 11
4. Sélection des Caractéristiques 15
III. Apprentissage automatique et entraînement des modèles 17
A. Présentation des algorithmes de Machine Learning utilisés 18
1. Random Forest 18
2. Naïve Bayes 18
3. Support Vector Machine (SVM) 19
4. K-Nearest Neighbors (KNN) 20
5. Régression Logistique 20
B. Méthodologie d'entraînement des modèles 21
C. Évaluation des performances 21
D. Comparaison des modèles 23
IV. Prototypage, résultats et analyse critique 24
A. Conception du prototype de détection d'intrusions 24
B. Fonctionnement du script de prédiction 24
CONCLUSION 27
BIBLIOGRAPHIE 28

REMERCIEMENTS

En premier lieu, nous remercions Dieu le très haut qui nous a donné le courage et la volonté de réaliser ce modeste travail. Nous tenons à saisir cette occasion et adresser nos profonds remerciements et nos profondes reconnaissances à toutes personnes qui nous ont aidé de près ou de loin dans la réalisation de ce projet.

Nous remercions DR. Moubarak, en tant que très chèr Encadrant de projet & Professeur, pour ses précieux conseils, sa disponibilité, sa compréhension, sa gentillesse, ses encouragements et son orientation tout au long de notre recherche.

Un grand merci à tous nos enseignants pour toutes les connaissances qu'ils nous ont inculquées tout au long de notre cursus universitaire. Enfin nous exprimons notre profonde reconnaissance à tous responsables de l'université de Djibouti qui ont contribué à notre formation.

INTRODUCTION

Dans un monde numérique en constante évolution, la sécurité des réseaux informatiques est devenue essentielle pour les entreprises, qu'elles soient petites ou grandes. Avec l'apparition de cyberattaques de plus en plus complexes et sophistiquées, il est devenu indispensable de mettre en place des solutions capables de protéger efficacement les systèmes d'information. C'est dans ce contexte que les systèmes de détection d'intrusion réseau (NIDS) basés sur le Machine Learning (ML) représentent une avancée importante. Ces systèmes ne se contentent pas seulement de détecter des attaques connues à partir de signatures prédéfinies, mais ils sont aussi capables d'apprendre, d'évoluer et de s'adapter face à de nouvelles menaces, assurant ainsi une protection plus intelligente et dynamique.

L'intérêt du Machine Learning en cybersécurité repose principalement sur sa capacité à analyser de grandes quantités de données en temps réel. Grâce à cela, il devient possible de repérer rapidement des comportements suspects ou malveillants au sein du trafic réseau. Contrairement aux méthodes traditionnelles, qui se basent uniquement sur des signatures d'attaques déjà connues, les modèles de ML peuvent identifier des anomalies sans avoir une connaissance préalable précise de l'attaque. Cela permet notamment de détecter des menaces récentes ou des attaques dites zero-day, qui ne sont pas encore répertoriées.

Notre projet, intitulé **Machine Learning : Système de Détection d'Intrusion Réseaux**, s'inscrit dans cette démarche moderne et innovante. Il a pour objectif de concevoir un modèle capable de différencier de manière fiable le trafic réseau normal des tentatives d'intrusion.

Ce projet revêt une importance particulière, car il contribue à renforcer la sécurité des infrastructures informatiques et à réduire les risques de dommages parfois très graves causés par les cyberattaques.

I. DESCRIPTION DU PROJET

Ce projet de **Machine Learning (Système de Détection d'Intrusion Réseaux)** représente pour nous une véritable immersion dans l'application du Machine Learning au domaine de la cybersécurité, dans le cadre de notre formation en informatique. À travers cette section, nous présentons les bases du projet, ses objectifs pédagogiques ainsi que la démarche méthodologique adoptée pour sa réalisation.

A. FONDEMENTS DU SYSTEME DE DETECTION D'INTRUSION RESEAUX

Système de Détection d'Intrusion Réseaux (NIDS) a pour rôle de surveiller en permanence le trafic circulant sur un réseau informatique afin de repérer des activités suspectes ou clairement malveillantes. De manière classique, ces systèmes s'appuient sur des signatures d'attaques connues ainsi que sur des méthodes d'analyse heuristique pour détecter les menaces.

Dans le cadre de notre projet, nous cherchons à comprendre comment les techniques de Machine Learning peuvent compléter et améliorer cette approche traditionnelle. L'idée est de permettre au système d'apprendre à partir des flux de données réseau et de reconnaître automatiquement des schémas ou comportements pouvant indiquer une tentative d'intrusion.

B. OBJECTIF EDUCATIF

Ce projet poursuit un double objectif. D'une part, il vise à renforcer notre compréhension pratique des applications du Machine Learning. D'autre part, il consiste à concevoir un modèle prototype capable d'identifier efficacement les intrusions réseau.

À travers cette initiative, nous souhaitons **:**

- **Développer des compétences techniques :** Approfondir notre compréhension du fonctionnement des systèmes de détection d'intrusion et analyser comment les algorithmes de Machine Learning peuvent améliorer leur performance.

C. APPROCHE MÉTHODOLOGIQUE

Notre démarche méthodologique suit une progression structurée, en cohérence avec les différentes parties présentées dans ce rapport. Elle s'articule autour de trois grandes phases **:** la préparation des données, l'apprentissage automatique et le prototypage.

1. Collecte et Prétraitement des Données

La première étape consiste à collecter les données à partir d'une source fiable, notamment le dataset **NSL-KDD**, recommandé pour les projets académiques en détection d'intrusion.

Après l'importation des fichiers dans l'environnement Python, une phase de prétraitement est réalisée afin de préparer les données pour l'entraînement des modèles. Cette phase comprend **:**

- La compréhension de la structure du dataset (types de variables, distribution des classes).
- Le nettoyage des données (suppression des doublons et gestion d'éventuelles incohérences).
- L'encodage des variables catégorielles (protocol_type, service, flag) en valeurs numériques.
- La standardisation des données pour améliorer les performances des modèles.
- La visualisation des classes (trafic normal vs attaque).
- La sélection des caractéristiques les plus pertinentes pour la détection.

Cette étape est essentielle car la qualité des données influence directement la performance des modèles.

2. Apprentissage Automatique et Entraînement des Modèles

Une fois les données préparées, nous procédons à l'entraînement de plusieurs algorithmes de Machine Learning supervisé **:**

- **Random Forest**
- **Naïve Bayes**
- **Support Vector Machine (SVM)**
- **KNN**
- **Régression Logistique**

Ces modèles sont entraînés à l'aide de la bibliothèque Scikit-Learn.

Les données sont divisées en ensembles d'entraînement et de test afin d'évaluer objectivement les performances.

L'évaluation des modèles repose sur **:**

- La matrice de confusion
- La précision (Accuracy)
- Le score F1
- Le rappel (Recall)

Une comparaison des performances est ensuite effectuée afin d'identifier le modèle le plus adapté à la détection d'intrusions.

3. Prototypage et Validation

La dernière phase consiste à développer un prototype fonctionnel permettant de tester le modèle entraîné sur de nouvelles données réseau.

Un script de prédiction est conçu afin de **:**

- Charger le modèle entraîné
- Recevoir une nouvelle entrée (ligne de données réseau)
- Générer une prédiction (Normal ou Attaque)

Ce prototype illustre concrètement le fonctionnement du système et permet de valider l'efficacité du modèle dans un contexte simulé.

Une fois les bases méthodologiques établies, la mise en œuvre technique débute par l'acquisition et la préparation rigoureuse des données, étape indispensable avant toute analyse.

II. COLLECTE DES DONNÉES

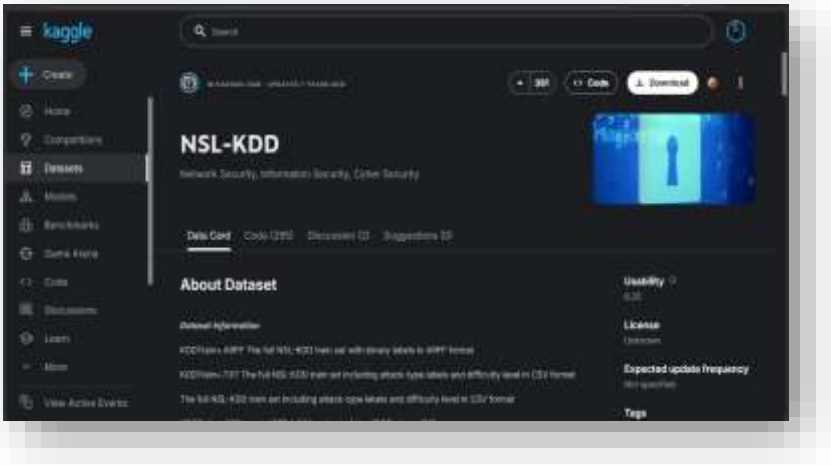
La phase de collecte des données constitue une étape essentielle dans la réalisation de notre projet de détection d'intrusion réseau basé sur le Machine Learning. En effet, la performance et la fiabilité de notre modèle dépendent directement de la qualité des données utilisées. Cette étape a consisté à rechercher et sélectionner un jeu de données pertinent, capable de représenter différents types de trafic réseau, afin d'entraîner et de tester efficacement notre modèle.

A. SOURCE DES DONNÉES

Pour disposer d'un ensemble de données complet et adapté à notre étude, nous avons utilisé la plateforme Kaggle. Il s'agit d'un espace en ligne très connu dans le domaine de la data science, qui met à disposition des jeux de données, des compétitions ainsi que de nombreuses ressources pédagogiques.

Grâce à Kaggle, nous avons pu accéder facilement à un dataset approprié pour notre projet, notamment le jeu de données NSL-KDD, largement utilisé dans les travaux académiques liés à la détection d'intrusions réseau.

Source : https://www.kaggle.com/datasets/sampadab17/network-intrusiondetection



B. IMPORTATION DES DONNÉES

Nous avons choisi un dataset spécialement dédié à la détection d'intrusions réseau, car il correspond parfaitement aux objectifs de notre projet. Ce jeu de données est à la fois varié et représentatif, ce qui nous permet de travailler sur différents types de situations, incluant du trafic normal ainsi que plusieurs formes d'attaques.

Le dataset est organisé en deux parties principales **:**

• **Ensemble d'entraînement :** Il contient des données simulées représentant à la fois un trafic réseau normal et des activités malveillantes. Cet ensemble est utilisé pour apprendre au modèle à reconnaître les différences entre un comportement normal et une tentative d'intrusion. Plus le modèle est exposé à des cas variés, plus il devient capable de détecter efficacement les anomalies.

• **Ensemble de test :** Cette partie est utilisée après l'entraînement afin d'évaluer les performances du modèle. Les données qu'elle contient n'ont pas été vues auparavant par le modèle. Cela nous permet de vérifier s'il est réellement capable de généraliser ses apprentissages et de fonctionner correctement face à de nouvelles situations, comme cela serait le cas dans un environnement réel.

C. PRÉTRAITEMENT DES DONNÉES

1. COMPRÉHENSION DES DONNÉES

Le prétraitement des données constitue une étape essentielle dans la réalisation de notre projet. Il permet de s'assurer que les informations utilisées pour entraîner le modèle de Machine Learning sont claires, structurées et adaptées à l'analyse. Avant toute transformation, il est important de bien comprendre la nature et la structure du dataset.

Après analyse, nous avons constaté que la base de données contient **43 colonnes** et **125 973 lignes**. Chaque colonne correspond à une caractéristique décrivant le trafic réseau, tandis qu'une colonne spécifique joue un rôle central dans notre étude.

La colonne cible est label. Elle indique si une connexion réseau correspond à un comportement normal ou à un type d'attaque particulier. Lorsque le trafic ne présente aucune activité malveillante, il est désigné par la valeur **"normal"**. Dans le cas contraire, la colonne précise le type d'attaque détecté.

Notre objectif est donc d'exploiter les autres colonnes du dataset afin de permettre au modèle de déterminer automatiquement si un trafic réseau est normal ou s'il s'agit d'une tentative d'intrusion.

```
train.info()
✓ 0.0s
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 125972 entries, 0 to 125971
Data columns (total 43 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   duration                     125972 non-null  int64
 1   protocol_type                125972 non-null  object
 2   service                      125972 non-null  object
 3   flag                         125972 non-null  object
 4   src_bytes                    125972 non-null  int64
 5   dst_bytes                    125972 non-null  int64
 6   land                         125972 non-null  int64
 7   wrong_fragment               125972 non-null  int64
 8   urgent                       125972 non-null  int64
 9   hot                          125972 non-null  int64
 10  num_failed_logins            125972 non-null  int64
 11  logged_in                    125972 non-null  int64
 12  num_compromised              125972 non-null  int64
 13  root_shell                   125972 non-null  int64
 14  su_attempted                 125972 non-null  int64
 15  num_root                     125972 non-null  int64
 16  num_file_creations           125972 non-null  int64
 17  num_shells                   125972 non-null  int64
 18  num_access_files             125972 non-null  int64
 19  num_outbound_cmds            125972 non-null  int64
...
 41  attack                       125972 non-null  object
 42  last_flag                    125972 non-null  int64
dtypes: float64(15), int64(24), object(4)
memory usage: 41.3+ MB
```

2. NETTOYAGE DES DONNEES

Le nettoyage des données constitue une étape cruciale dans la préparation du dataset pour l'apprentissage automatique. En effet, la qualité des données utilisées influence directement la performance et la fiabilité des modèles

Dans notre projet, nous avons appliqué plusieurs opérations de nettoyage :

Gestion des Valeurs Manquantes : bien que le dataset NSL-KDD ne contienne pas de valeurs nulles, nous avons mis en place une vérification systématique pour garantir la robustesse du modèle face à d'éventuelles données incomplètes dans des utilisations futures.

Suppression des Caractéristiques Redondantes : Nous avons identifié et éliminé les colonnes ne contenant que des valeurs constantes ou peu variées, qui n'apportent pas d'information significative pour la détection.

```
Valeurs manquantes dans les données d'entrainement:
 duration                     0
protocol_type                 0
service                       0
flag                          0
src_bytes                     0
dst_bytes                     0
land                          0
wrong_fragment                0
urgent                        0
hot                           0
num_failed_logins             0
logged_in                     0
num_compromised               0
root_shell                    0
su_attempted                  0
num_root                      0
num_file_creations            0
num_shells                    0
num_access_files              0
num_outbound_cmds             0
is_host_login                 0
is_guest_login                0
count                         0
srv_count                     0
serror_rate                   0
srv_serror_rate               0
rerror_rate                   0
srv_rerror_rate               0
same_srv_rate                 0
diff_srv_rate                 0
```

1. ENCODAGE DE DONNEES CATEGORIELLES EN NUMERIQUE

Afin de rendre le dataset exploitable par les algorithmes de Machine Learning, nous avons procédé à deux transformations essentielles :

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...
0	0	2	44	9	146	0	0	0	0	0	...
1	0	1	49	5	0	0	0	0	0	0	...
2	0	1	24	9	232	8153	0	0	0	0	...
3	0	1	24	9	199	420	0	0	0	0	...
4	0	1	49	1	0	0	0	0	0	0	...

5 rows × 43 columns

- **Encodage des variables catégorielles** : les colonnes **protocol_type, service** et **flag**, qui contiennent des valeurs textuelles (par exemple ***tcp, udp, http, SF***), ont été converties en valeurs numériques grâce à un encodage. Cette étape permet aux modèles de traiter ces informations de manière quantitative.
- **Création d'une cible binaire** : pour simplifier la tâche de classification, nous avons regroupé toutes les attaques sous une seule catégorie. Ainsi, une nouvelle colonne attack_binary a été ajoutée, prenant la valeur 0 pour le trafic normal et 1 pour tout type d'attaque. Cette transformation permet de passer d'une classification multi-classes (plusieurs types d'attaques) à une classification binaire (normal vs attaque), ce qui est plus adapté à la détection d'intrusions.

2. STANDARDISATION DES DONNÉES

La standardisation permet de mettre toutes les variables du dataset sur la même échelle. Certaines colonnes, comme src_bytes, peuvent avoir des valeurs très grandes, alors que d'autres, comme serror_rate, sont limitées entre 0 et 1. Sans normalisation, les modèles risquent de donner plus d'importance aux variables avec de grandes valeurs.

Pour éviter cela, nous avons utilisé **StandardScaler** de Scikit-Learn. Cette méthode ajuste chaque colonne pour qu'elle ait une moyenne de 0 et un écart-type de 1. Ainsi, toutes les caractéristiques contribuent de manière équilibrée à l'apprentissage.

Cette étape est particulièrement utile pour les algorithmes sensibles aux différences d'échelle, comme le **SVM** ou le **KNN**. Nous avons appliqué le scaler sur l'ensemble d'entraînement, puis utilisé les mêmes paramètres pour transformer l'ensemble de test, afin de garantir une évaluation correcte.

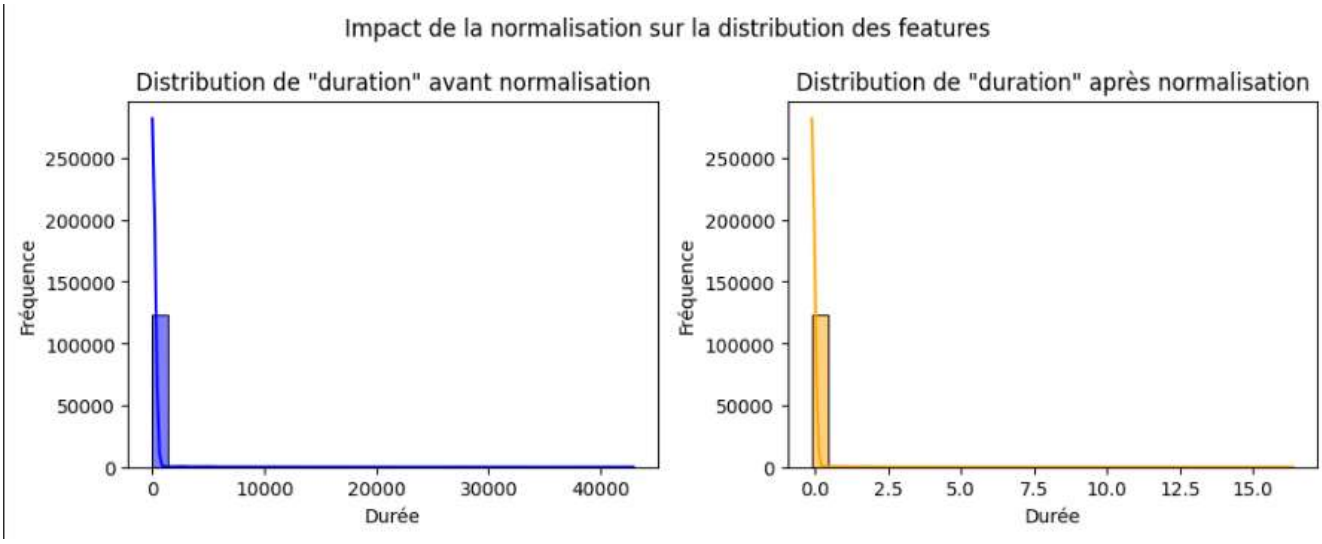
```python
# BLOC 13 : NORMALISATION DES VARIABLES NUMÉRIQUES
# Sauvegarde du dataset original avant normalisation (pour comparaison)
train_original = train.copy()

# Identification des colonnes numériques (hors cible et catégorielles)
numerical_cols = [col for col in train.columns if col not in ['attack'] + categorical_cols]

# Calcul des paramètres de normalisation sur les données d'entraînement uniquement
scaler = StandardScaler().fit(train[numerical_cols])
means  = train[numerical_cols].mean()
stds   = train[numerical_cols].std()

# Application de la normalisation sur train et test
train[numerical_cols] = (train[numerical_cols] - means) / stds
test[numerical_cols]  = (test[numerical_cols]  - means) / stds

train.head()
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_di
0	-0.110249	2	44	9	-0.007737	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	...	-1.161030	
1	-0.110249	1	49	5	-0.007762	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	...	-0.938288	
2	-0.110249	1	24	9	-0.007723	-0.002891	-0.014089	-0.089486	-0.007736	-0.095076	...	1.066389	
3	-0.110249	1	24	9	-0.007728	-0.004814	-0.014089	-0.089486	-0.007736	-0.095076	...	1.066389	
4	-0.110249	1	49	1	-0.007762	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	...	-1.005111	

5 rows × 43 columns

Nous avons utilisé la fonction **StandardScaler** pour mettre à l'échelle notre ensemble de données d'entraînement. Cette méthode calcule la moyenne et l'écart-type de chaque variable, puis les utilise pour transformer les valeurs afin qu'elles soient centrées autour de 0 et normalisées.

Les mêmes paramètres (moyenne et écart-type) ont ensuite été appliqués à l'ensemble de test, ce qui garantit une cohérence entre les deux ensembles et évite toute fuite d'information. Grâce à cette étape, toutes les caractéristiques du dataset sont sur une même échelle et peuvent être utilisées efficacement par les modèles de Machine Learning.

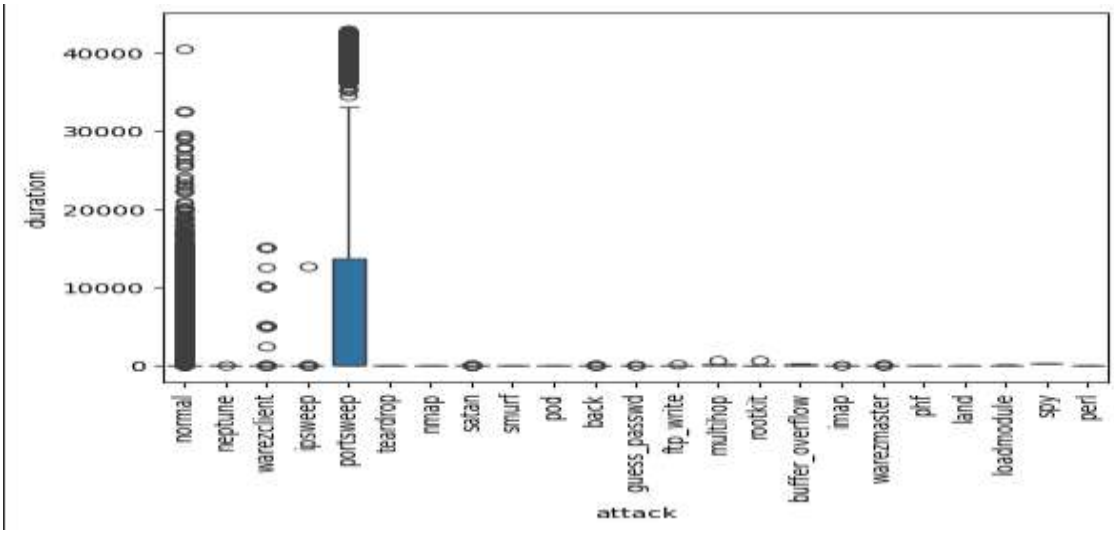


3. VISUALISATION DES DONNÉES

Nous avons généré plusieurs visualisations pour mieux comprendre la relation entre les caractéristiques du dataset et la variable cible. Ces graphiques nous ont permis :

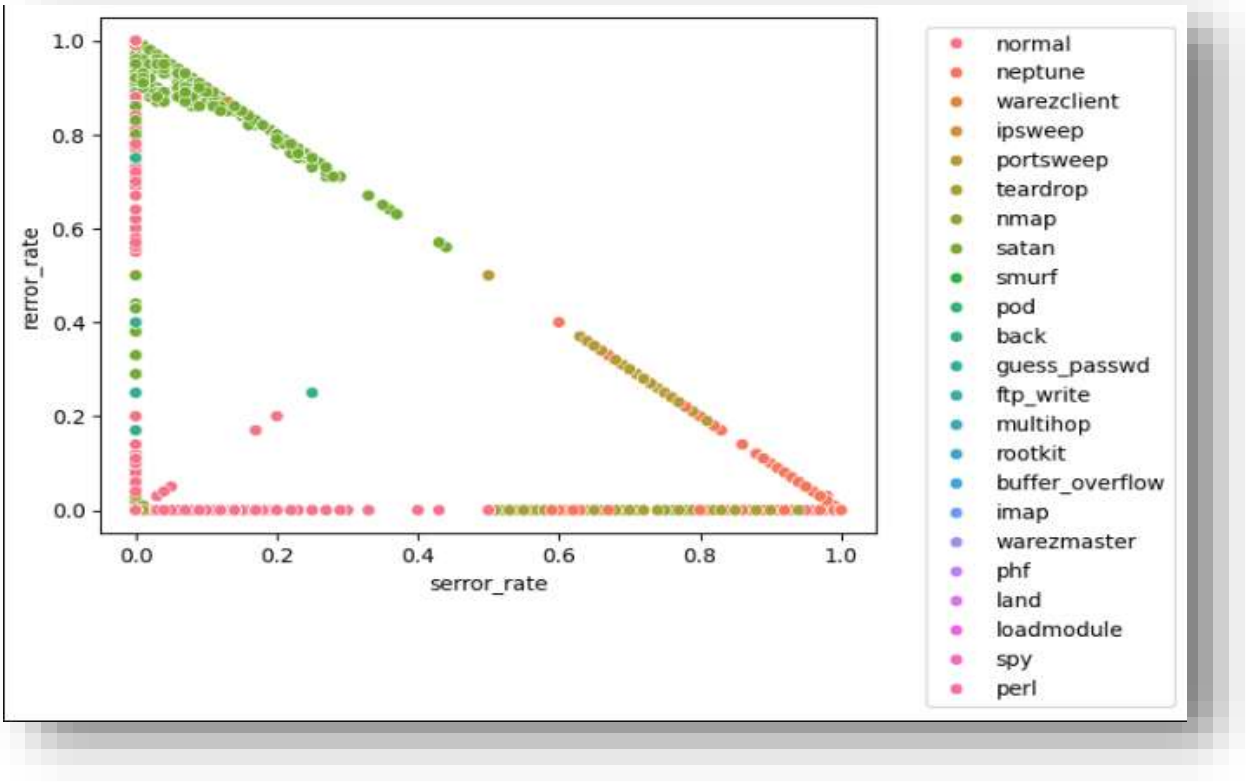
- d'identifier la répartition entre trafic normal et attaques,
- de repérer les types d'attaques les plus fréquents,
- de mettre en évidence certains schémas ou anomalies dans les données.

Ces représentations visuelles facilitent l'analyse et nous aident à sélectionner les caractéristiques les plus pertinentes pour l'entraînement des modèles.



Graphique de la durée des connexions : Le diagramme de la variable duration illustre la durée des connexions réseau pour différents types d'attaques. On observe que la majorité des connexions ont une durée très courte, souvent proche de zéro, ce qui montre que de nombreuses activités réseau, notamment certaines attaques, se produisent sur une période brève.

Cependant, certaines connexions présentent des durées plus longues, visibles sous forme de valeurs extrêmes. Les connexions normales apparaissent plus variées, tandis que plusieurs types d'attaques se concentrent autour de durées faibles.

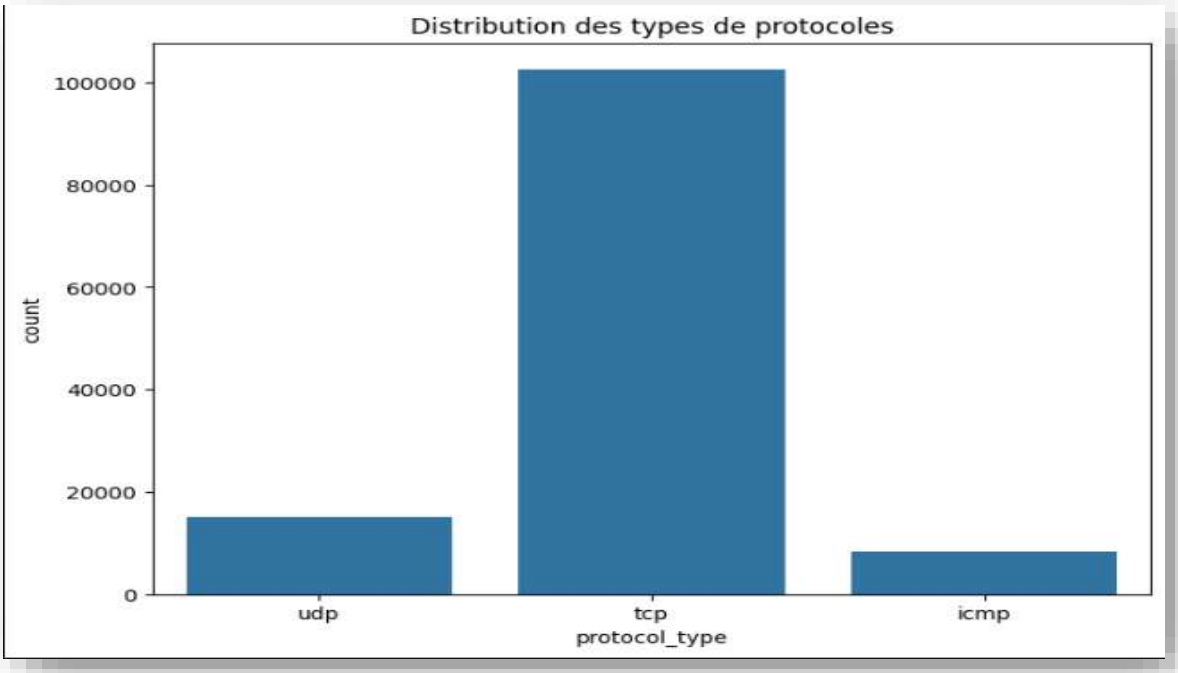


Ce nuage de points compare deux indicateurs liés aux erreurs dans le réseau : **serror_rate** et **rerror_rate**. Les points sont colorés selon la classe (0 pour trafic normal, 1 pour attaque).

On observe que :

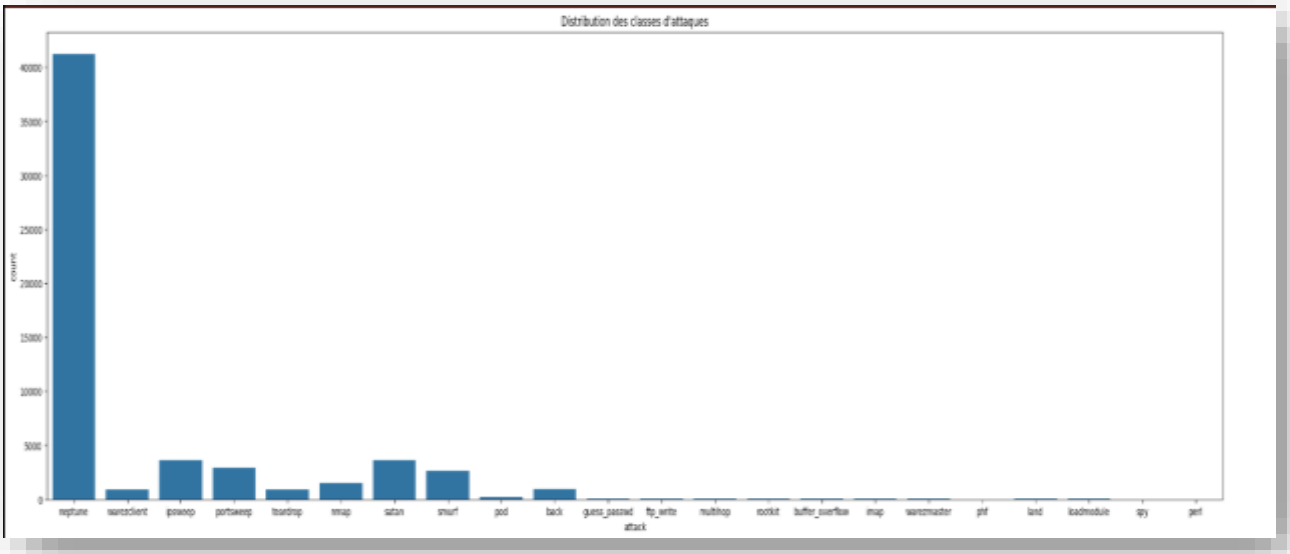
- Les **connexions normales (classe 0)** sont regroupées autour de faibles valeurs d'erreurs.
- **Les attaques (classe 1)** présentent une distribution différente, souvent corrélée entre les deux taux, ce qui crée une tendance diagonale dans le graphique.

Cette visualisation montre que ces deux variables sont pertinentes pour distinguer le trafic normal des intrusions.



Ce graphique montre combien de fois chaque type de protocole apparaît dans le dataset. On constate que le protocole **TCP** est de loin le plus utilisé, avec une fréquence largement supérieure aux autres. Les protocoles **UDP** et **ICMP** sont présents, mais beaucoup moins représentés.

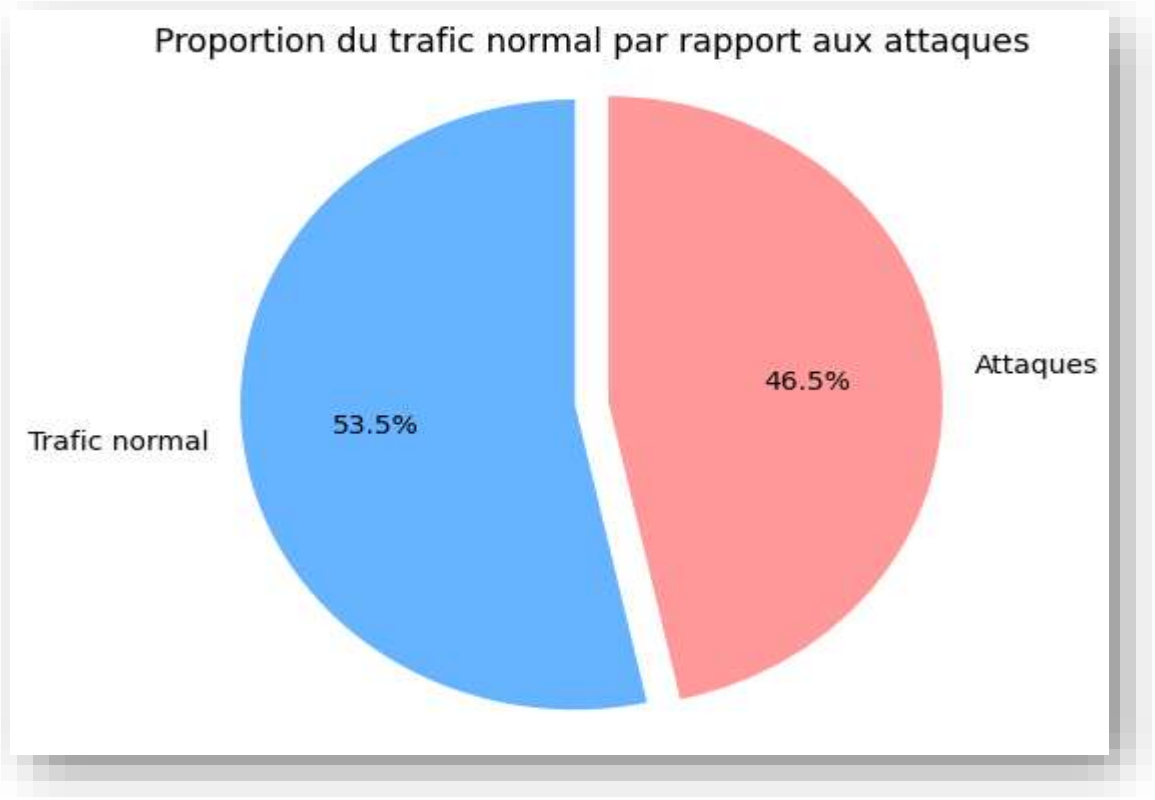
Cette répartition met en évidence la dominance du protocole TCP dans le trafic réseau, ce qui peut influencer l'entraînement et la détection des intrusions par les modèles de Machine Learning.



Ce graphique à barres illustre la fréquence des différents types d'attaques présentes dans le dataset. On observe que l'attaque **Neptune** est de loin la plus courante, suivie par d'autres attaques moins représentées.

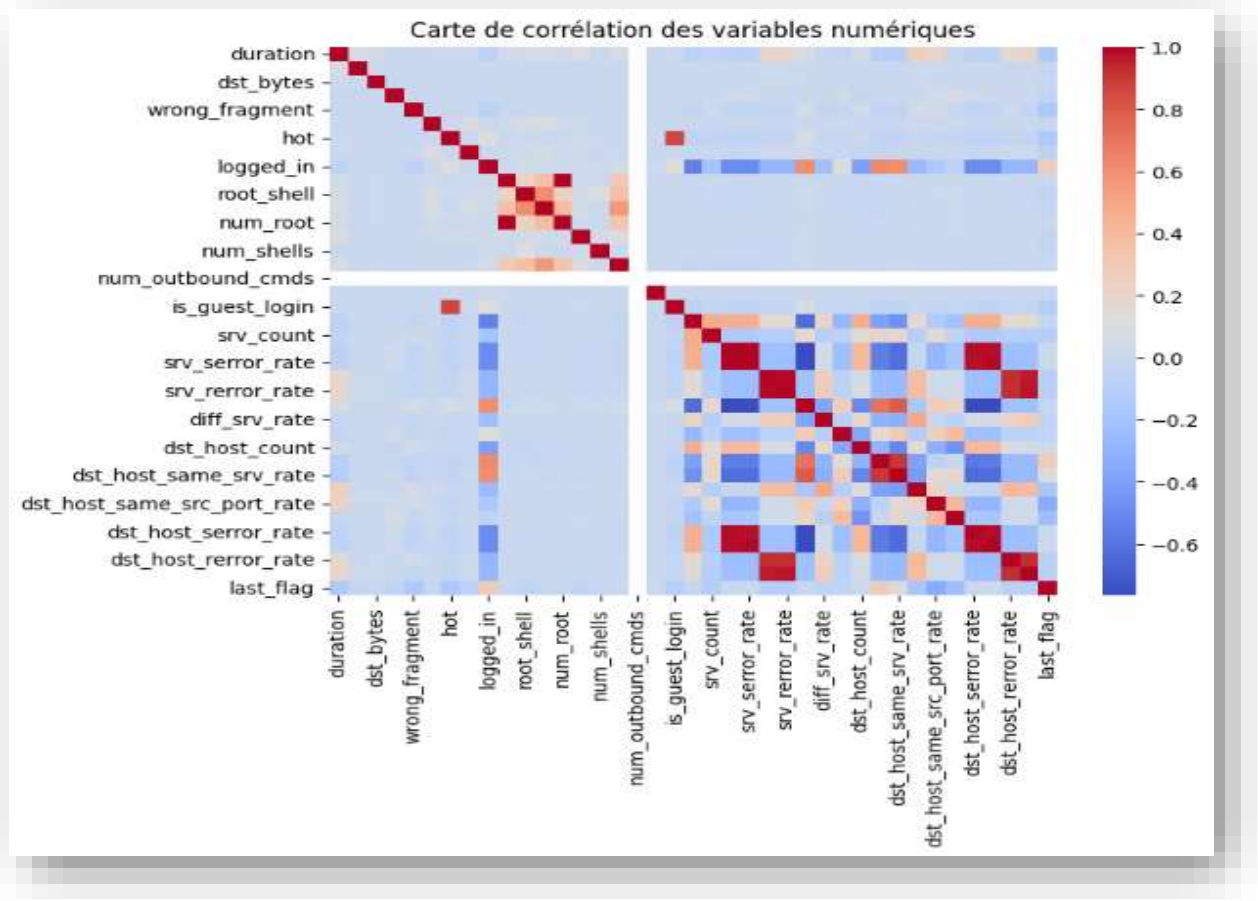
En parallèle, une quantité importante de trafic **normal** est également présente, ce qui permet de comparer directement les comportements légitimes aux activités malveillantes.

Cette visualisation met en évidence la dominance de certaines attaques dans le dataset et souligne l'importance de prendre en compte ce déséquilibre lors de l'entraînement des modèles de détection.



Ce graphique en secteurs illustre la répartition entre trafic normal et attaques. On observe que la moitié du cercle correspond au trafic **normal**, tandis que l'autre moitié regroupe l'ensemble des **attaques détectées**.

Cette représentation met en évidence l'équilibre relatif entre connexions normales et malveillantes dans le dataset, ce qui est important à prendre en compte lors de l'entraînement des modèles de détection.



Heatmap de corrélation :

La heatmap de corrélation présente les relations entre les différentes caractéristiques du dataset. Chaque case du tableau indique le degré de corrélation entre deux variables.

- Les **couleurs chaudes** (rouge/orange) traduisent une forte corrélation, c'est-à-dire que les deux variables évoluent souvent ensemble.
- Les **couleurs froides** (bleu) indiquent une faible corrélation ou l'absence de lien significatif.

Cette visualisation permet d'identifier rapidement les variables redondantes ou fortement liées, ce qui facilite la sélection des caractéristiques les plus pertinentes pour l'entraînement des modèles.

4. SÉLECTION DES CARACTÉRISTIQUES

La première étape a consisté à supprimer la colonne ne contenant que des zéros, car elle n'apportait aucune information utile. Ensuite, nous avons appliqué différentes méthodes pour identifier les caractéristiques les plus pertinentes du dataset.

Ces approches nous ont permis de réduire la complexité des données et de conserver uniquement les variables les plus significatives pour l'entraînement des modèles de détection.

```
[2]: train_df = train_df.drop(columns=['num_outbound_cmds'])
     test_df = test_df.drop(columns=['num_outbound_cmds'])
     train_df.info()
     train_df.head()
```

Pour identifier les variables les plus pertinentes dans la prédiction de la cible , nous avons utilisé le modèle ExtraTreesClassifier. Cette méthode repose sur l'évaluation de l'importance des caractéristiques par un ensemble d'arbres de décision.

Les étapes suivies sont les suivantes :

• **Entraînement du modèle** : ExtraTreesClassifier a été instancié avec 100 arbres de décision () et entraîné sur les données d'entraînement, en excluant la colonne .

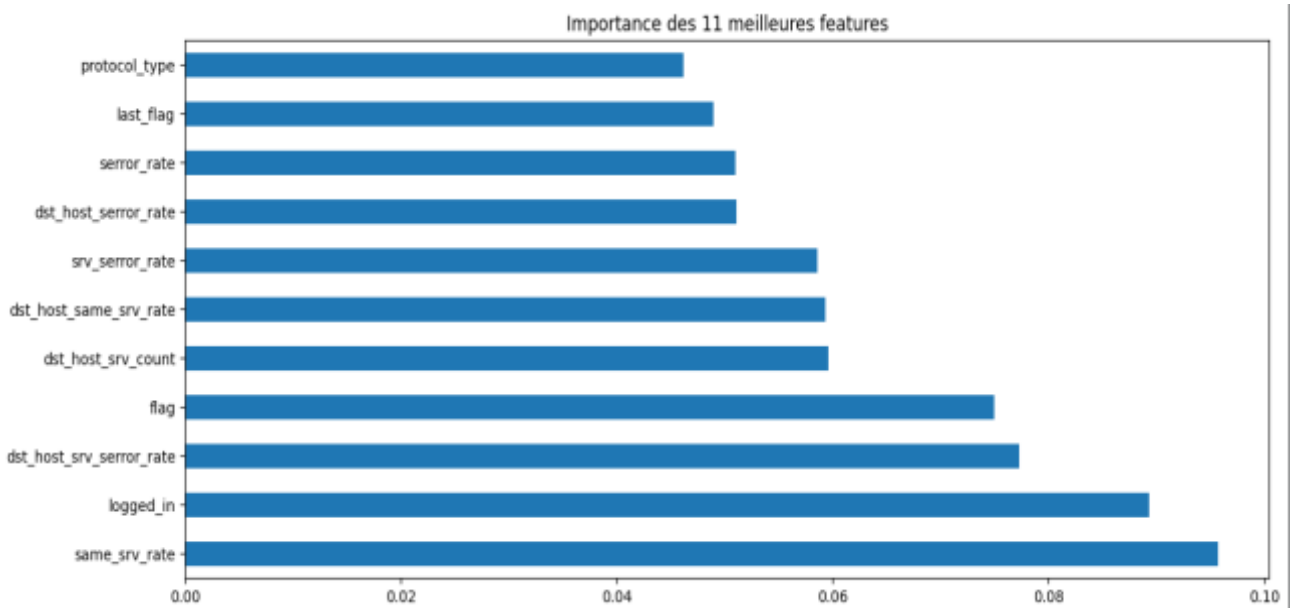
• **Calcul de l'importance des caractéristiques** : Après l'entraînement, la méthode a permis de mesurer la contribution de chaque variable à la prédiction de la cible.

• **Sélection des meilleures variables** : Les caractéristiques ont été triées par ordre d'importance, et les 11 plus significatives ont été retenues pour l'entraînement des modèles.

• **Visualisation :** Les résultats ont été présentés sous forme de diagramme à barres horizontal, facilitant la compréhension de l'importance relative de chaque caractéristique dans la détection des attaques.

Cette approche permet de réduire la complexité du dataset tout en conservant les variables les plus informatives, améliorant ainsi la performance et l'efficacité des modèles de Machine Learning.

ExtraTreesClassifier : Un modèle qui nous permet d'estimer automatiquement l'importance des variables grâce à un ensemble d'arbres de décision.



Après avoir identifié les variables les plus pertinentes grâce au modèle ExtraTreesClassifier, nous avons construit un nouveau DataFrame contenant uniquement ces caractéristiques retenues.

Ce sous-ensemble de données servira de base pour l'entraînement et l'évaluation des modèles de Machine Learning, en garantissant que seules les informations les plus significatives sont exploitées

Impact du prétraitement:

Le prétraitement a permis de transformer les données brutes en un ensemble structuré et optimisé pour l'entraînement des modèles de Machine Learning. Cette étape a consisté à :

- **Nettoyer les données** pour éliminer les valeurs inutiles ou redondantes,
- **Normaliser** les variables afin de les mettre sur une même échelle,
- **Sélectionner stratégiquement** les caractéristiques les plus significatives pour la prédiction.

```
[17]: # BLOC 17 : RÉDUCTION DU DATASET AUX FEATURES SÉLECTIONNÉES
train = train[top_features.tolist() + ['attack']]
test  = test[top_features.tolist()  + ['attack']]

train.head()
```

[17]:

	logged_in	same_srv_rate	dst_host_srv_count	serror_rate	flag	dst_host_same_srv_rate	srv_serror_rate	dst_host_srv_serror_rate	protocol_type	last_flag	dst_host_s
0	-0.809264	-1.321415	-1.035689	-0.637210	9	-1.161030	-0.631930	-0.624872	2	-1.965539	
1	-0.809264	-1.389655	-0.809859	1.602649	5	-0.938288	1.605089	1.618940	1	-0.219967	
2	1.235681	0.771285	1.258741	-0.189238	9	1.066389	-0.184526	-0.602434	1	0.652820	
3	1.235681	0.771285	1.258741	-0.637210	9	1.066389	-0.631930	-0.624872	1	0.652820	
4	-0.809264	-1.139441	-0.873091	-0.637210	1	-1.005111	-0.631930	-0.624872	1	0.652820	

Grâce à ce processus**, le dataset est devenu plus cohérent**, **plus lisible et mieux adapté à l'apprentissage automatique**, ce qui améliore la performance et la fiabilité des modèles.

Maintenant que les données sont nettoyées, standardisées et que les caractéristiques les plus pertinentes ont été sélectionnées, nous pouvons passer à la phase de modélisation en testant différents algorithmes d'apprentissage automatique.

III. APPRENTISSAGE AUTOMATIQUE ET ENTRAINEMENT DES MODELES

Après avoir préparé et prétraité les données, l'étape suivante consiste à **appliquer des algorithmes de Machine Learning afin de détecter automatiquement les intrusions réseau**.

Dans ce projet, plusieurs modèles de classification supervisée ont été testés afin d'identifier les connexions malveillantes dans le trafic réseau.

L'objectif principal est de construire un modèle capable de distinguer deux types de trafic :

- **Trafic normal (0)**
- **Trafic malveillant / attaque (1)**

Plusieurs algorithmes ont été sélectionnés afin de comparer leurs performances et déterminer lequel est le plus efficace pour la détection des intrusions.

A. PRESENTATION DES ALGORITHMES DE MACHINE LEARNING UTILISES

Dans ce projet, trois algorithmes de classification ont été utilisés : Random Forest, Naïve Bayes et Support Vector Machine (SVM).

Ces modèles ont été choisis pour leur **efficacité et leur capacité à traiter des problèmes de classification** dans des ensembles de données complexes.

1. RANDOM FOREST

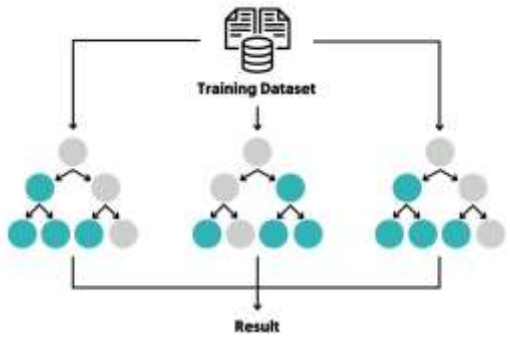
Le modèle Random Forest est un algorithme d'apprentissage supervisé basé sur un ensemble d'arbres de décision.

Au lieu d'utiliser un seul arbre de décision, Random Forest construit plusieurs arbres et combine leurs résultats pour produire une prédiction finale.

Chaque arbre est entraîné sur une partie aléatoire des données, ce qui permet de réduire le risque de surapprentissage et d'améliorer la robustesse du modèle.

Ce modèle est particulièrement efficace pour les problèmes de classification et fonctionne bien avec des datasets comportant un grand nombre de variables.

Dans le contexte de la détection d'intrusion réseau, Random Forest permet d'identifier les caractéristiques les plus importantes qui influencent la classification entre trafic normal et attaque.



2. NAÏVE BAYES

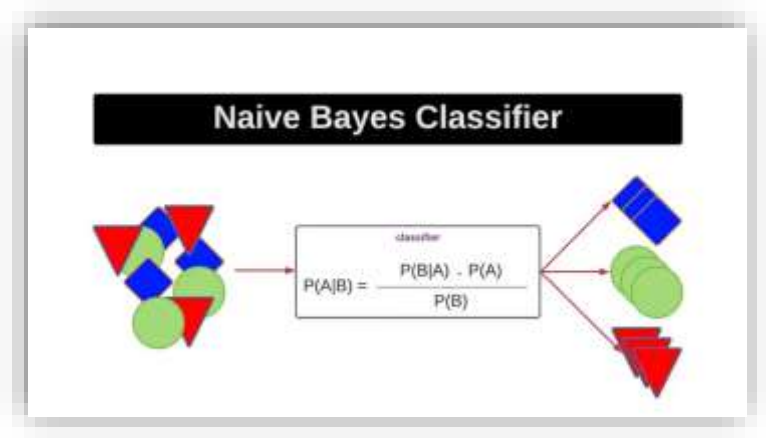
Le modèle Naïve Bayes est un algorithme de classification basé sur le théorème de Bayes.

Il repose sur l'hypothèse que les variables explicatives sont indépendantes les unes des autres.

Malgré cette hypothèse simplificatrice, Naïve Bayes est souvent très performant pour les problèmes de classification et présente plusieurs avantages :

- simplicité de mise en œuvre
- rapidité d'entraînement
- efficacité sur de grands volumes de données

Dans ce projet, l'algorithme Gaussian Naïve Bayes a été utilisé pour prédire si une connexion réseau correspond à une activité normale ou à une attaque.



3. SUPPORT VECTOR MACHINE (SVM)

Le modèle Support Vector Machine (SVM) est un algorithme de classification qui cherche à trouver la meilleure frontière de séparation entre les différentes classes.

Cette frontière, appelée hyperplan, permet de séparer les données appartenant aux différentes catégories avec la plus grande marge possible.

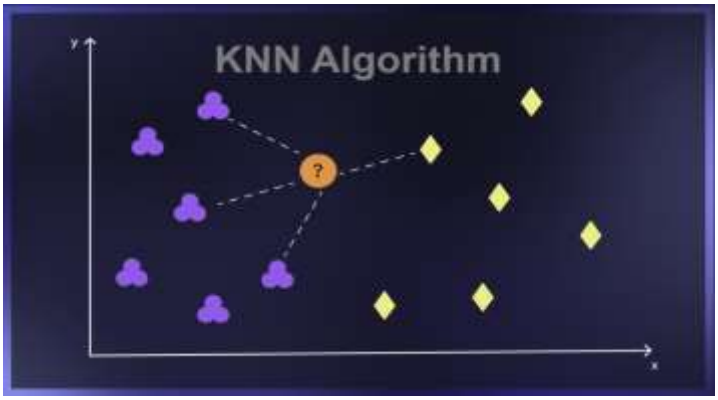
Le SVM est particulièrement efficace pour les problèmes de classification dans des espaces de grande dimension et peut gérer des relations complexes entre les variables.

Dans ce projet, le SVM est utilisé pour déterminer si une connexion réseau doit être classée comme trafic normal ou comme tentative d'intrusion.

4. K-NEAREST NEIGHBORS (KNN)

L'algorithme **K-Nearest Neighbors (KNN)** est une méthode de classification supervisée basée sur la similarité entre les données. Contrairement à d'autres modèles, KNN ne construit pas un modèle explicite lors de la phase d'apprentissage. Il mémorise simplement les données d'entraînement et effectue les prédictions en se basant sur les exemples les plus proches.

Lorsqu'une nouvelle donnée est introduite, l'algorithme calcule la distance entre cette donnée et toutes les observations du dataset d'entraînement. Il sélectionne ensuite les K voisins les plus proches, puis attribue la classe majoritaire parmi ces voisins.

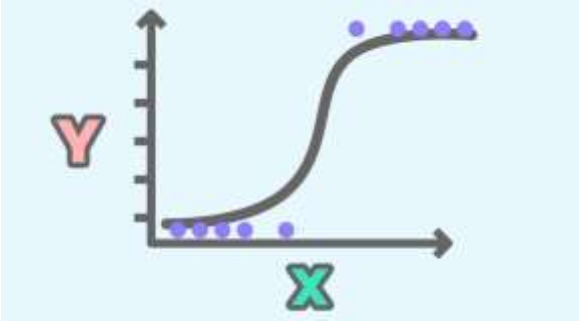


Dans le cadre de ce projet de détection d'intrusion, **KNN** permet de classer une connexion réseau en fonction de sa similarité avec des connexions déjà observées. Si la majorité des voisins proches correspondent à des attaques, alors la nouvelle connexion sera classée comme une attaque.

5. REGRESSION LOGISTIQUE

La **régression logistique** est un algorithme de classification supervisée largement utilisé pour prédire une variable binaire. Contrairement à la régression linéaire, elle ne prédit pas une valeur continue, mais la probabilité d'appartenance à une classe donnée.

Dans ce modèle, les données d'entrée sont combinées à l'aide d'une fonction linéaire, puis transformées par une fonction sigmoïde. Cette fonction permet d'obtenir une **probabilité comprise entre 0 et 1**. En fonction de cette probabilité, la donnée est ensuite classée comme attaque ou trafic normal.



Dans le cadre de ce projet de détection d'intrusion, la régression logistique est utilisée comme modèle de base afin d'évaluer la capacité de séparation des données. Elle permet de déterminer si les connexions réseau peuvent être distinguées de manière linéaire en fonction de leurs caractéristiques.

B. MÉTHODOLOGIE D'ENTRAÎNEMENT DES MODÈLES

Chaque modèle a été entraîné sur notre ensemble de données préalablement normalisé, puis évalué sur l'ensemble de test afin de mesurer sa capacité de généralisation**. Pour évaluer les performances des différents modèles, nous avons utilisé l'accuracy comme métrique principale.** Cette métrique permet d'obtenir une estimation directe de la proportion de prédictions correctes réalisées par un modèle.

Accuracy : L'accuracy est une métrique d'évaluation utilisée en Machine Learning pour mesurer la proportion de prédictions correctes effectuées par un modèle par rapport au nombre total de prédictions.

C. ÉVALUATION DES PERFORMANCES

Après l'entraînement des différents modèles de Machine Learning, il est nécessaire d'évaluer leurs performances afin de mesurer leur capacité à détecter correctement les attaques réseau. Pour cela, plusieurs métriques d'évaluation ont été utilisées, notamment la matrice de confusion et le score F1.

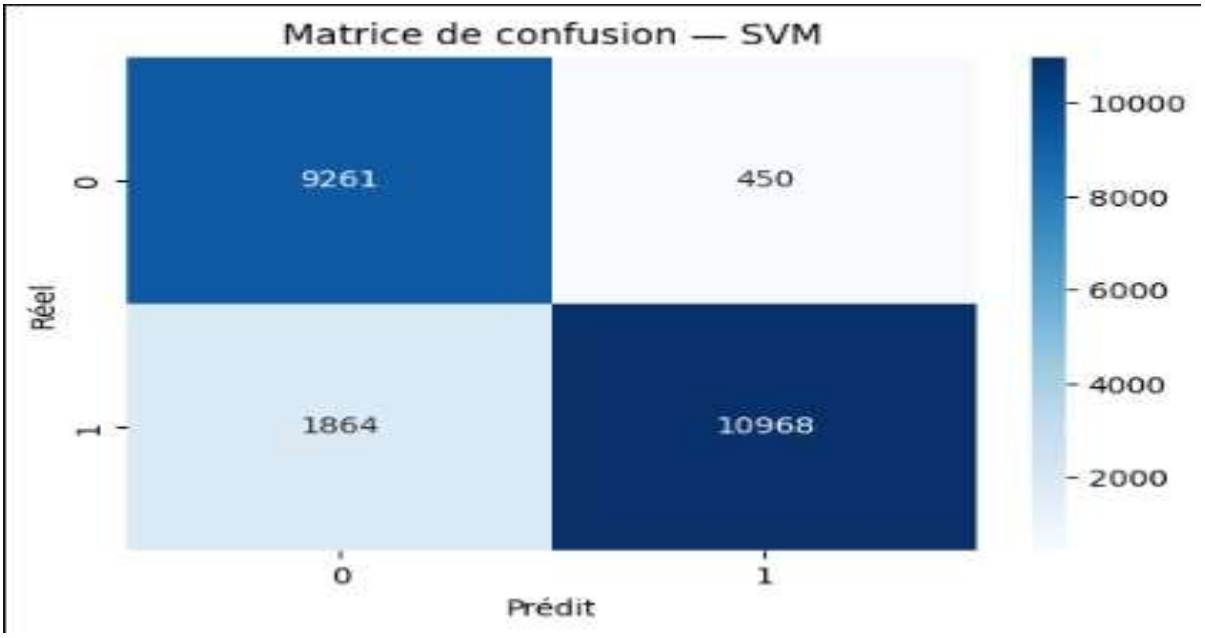
- Matrice de confusion :

La matrice de confusion est un outil permettant d'analyser en détail les performances d'un modèle de classification. Elle compare les valeurs réelles aux valeurs prédites par le modèle, ce qui permet d'identifier les classifications correctes ainsi que les erreurs de prédiction.

Dans le cadre de la détection d'intrusion, la matrice de confusion permet d'observer :

- les connexions normales correctement classées
- les attaques correctement détectées
- les fausses alertes (connexions normales classées comme attaques)

- les attaques non détectées par le modèle



La matrice de confusion du modèle SVM montre que le modèle **est capable de détecter correctement un grand nombre de connexions normales (9261) ainsi qu'un nombre élevé d'attaques (10968)**. Cependant, on observe également 450 faux positifs, correspondant à des connexions normales classées à tort comme des attaques, et 1864 faux négatifs, correspondant à des attaques non détectées.

Ces résultats indiquent que le modèle est globalement performant, mais qu'il reste des améliorations possibles, notamment pour réduire le nombre d'attaques non détectées, ce qui constitue un enjeu critique dans un système de détection d'intrusion.

- Score F1

Le score F1 est une métrique d'évaluation qui combine deux mesures importantes :

- **la précision (precision) :** proportion des prédictions positives correctes
- **le rappel (recall) :** capacité du modèle à détecter les attaques

Le score F1 permet d'obtenir un équilibre entre ces deux mesures et est particulièrement utile lorsque les données sont déséquilibrées. Dans le contexte de la détection d'intrusion réseau, cette métrique permet d'évaluer la capacité du modèle à détecter efficacement les attaques tout en limitant les erreurs de classification.

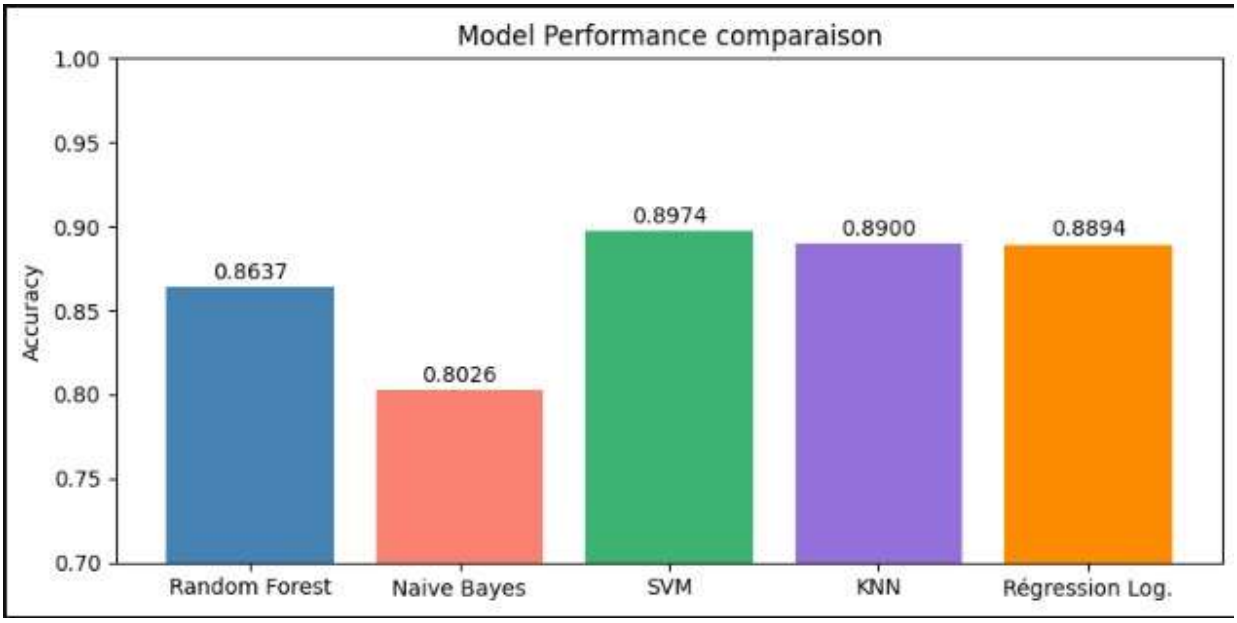
Modèle	Accuracy	F1-score
Random Forest	0.8628	0.85
Naïve Bayes	0.8063	0.81
SVM	0.8974	0.89
KNN	0.8900	0.88
Régression Logistique	0.8894	0.87

Les résultats obtenus montrent que le modèle **SVM présente les meilleures performances globales avec un score F1 proche de 0.9**, indiquant un bon équilibre entre précision et rappel. Les modèles KNN et Régression Logistique offrent également de très bonnes performances avec des scores F1 respectifs de 0.88 et 0.87, confirmant leur capacité à bien généraliser sur les données.

D. COMPARAISON DES MODELES

Afin d'identifier le modèle le plus performant pour la détection des intrusions, une comparaison des différents algorithmes utilisés a été réalisée. Les modèles étudiés dans ce projet sont :

- **Random Forest**
- **Naïve Bayes**
- **Support Vector Machine (SVM)**
- **KNN (K-Nearest Neighbors)**
- **Régression Logistique**



Le graphique de comparaison montre que **le modèle SVM obtient la meilleure performance globale avec la plus grande accuracy.** Les modèles KNN et régression logistique présentent également de très bons résultats, proches de celui du SVM. En revanche, Random Forest est légèrement moins performant, tandis que Naïve Bayes affiche les résultats les plus faibles.

IV. PROTOTYPAGE, RESULTATS ET ANALYSE CRITIQUE

A. CONCEPTION DU PROTOTYPE DE DETECTION D'INTRUSIONS

Dans cette phase, nous avons développé un prototype de système de détection d'intrusions basé sur le Machine Learning. L'objectif est de mettre en pratique les modèles entraînés à travers une application interactive.

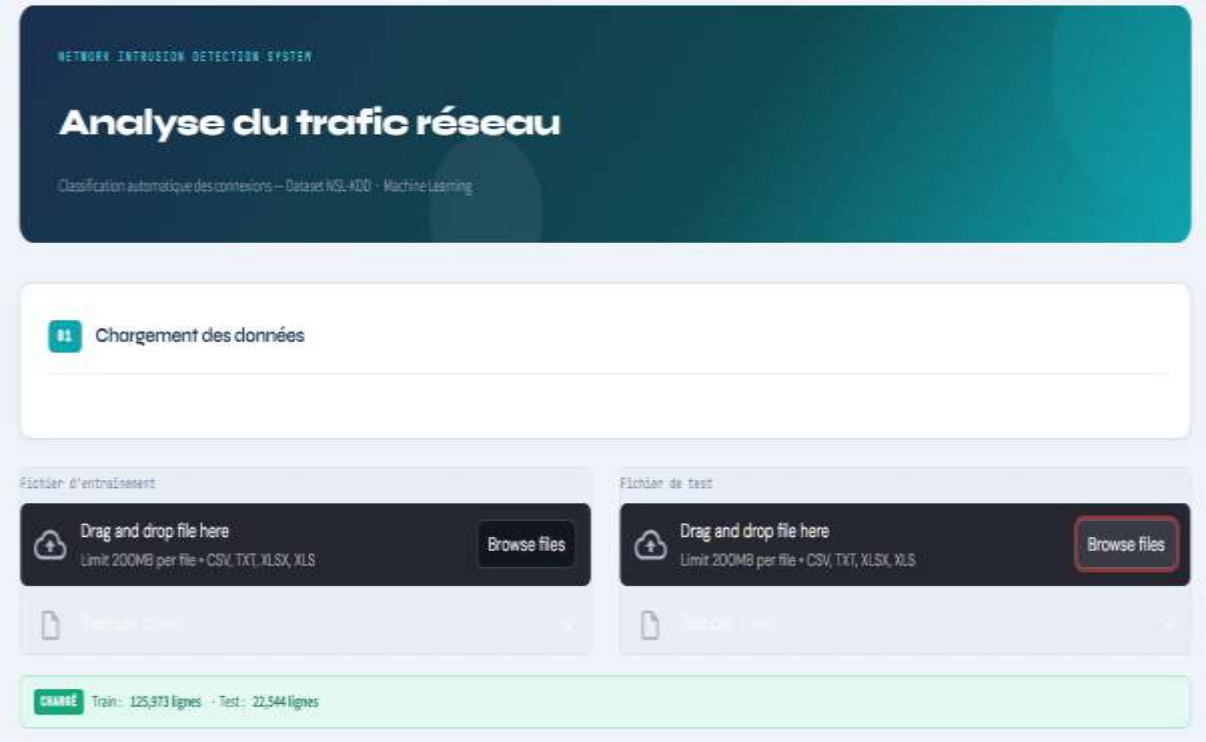
Pour cela, nous avons utilisé Python avec la bibliothèque Streamlit afin de créer une interface simple permettant de charger les données, entraîner les modèles et effectuer des prédictions.

Le prototype est structuré en plusieurs étapes : chargement des données, prétraitement automatique, entraînement des modèles et prédiction. Cette organisation permet de suivre facilement le fonctionnement du système.

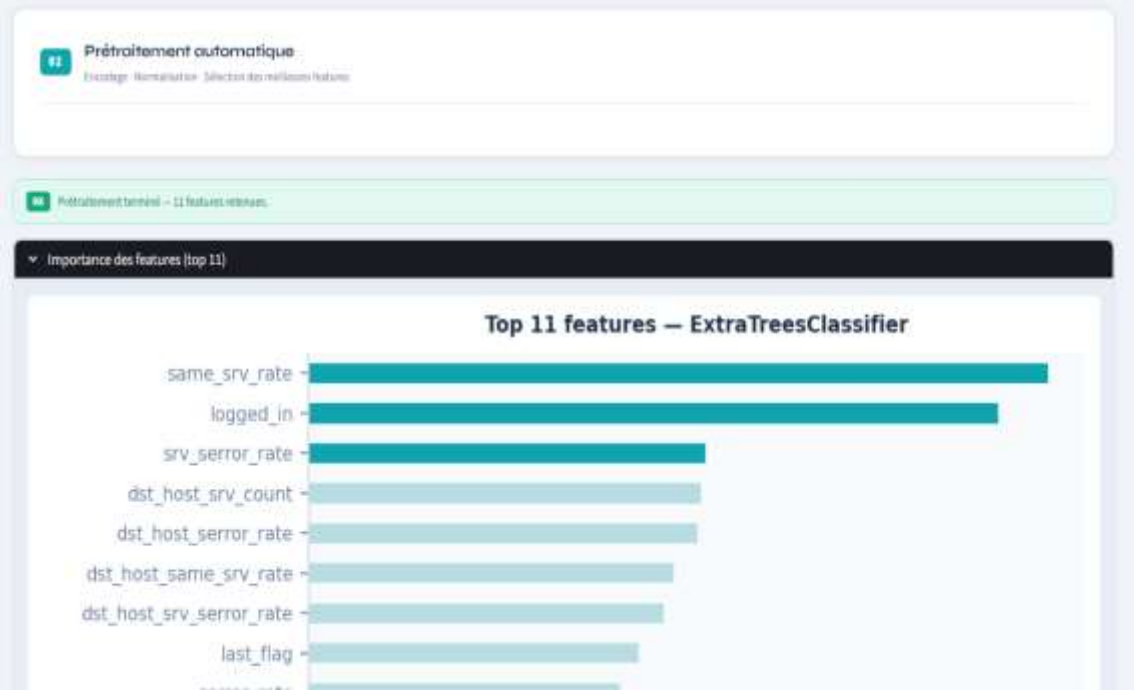
B. FONCTIONNEMENT DU SCRIPT DE PREDICTION

Le prototype fonctionne en plusieurs étapes **:**

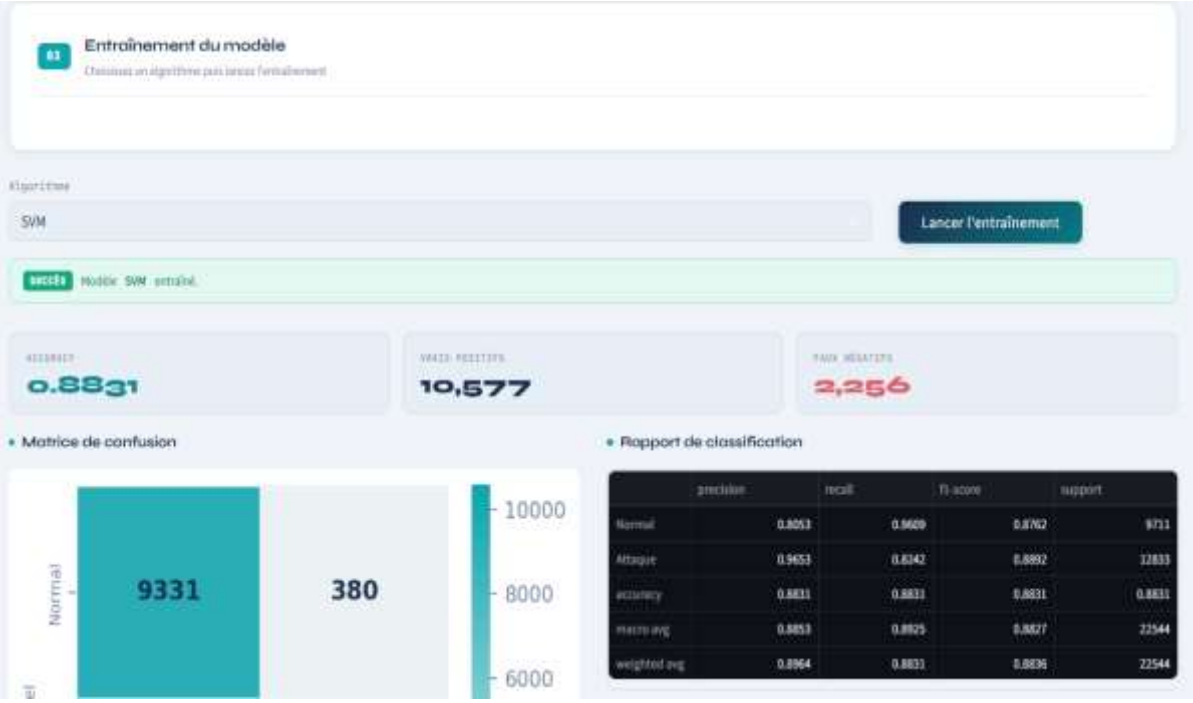
- Chargement des fichiers d'entraînement et de test



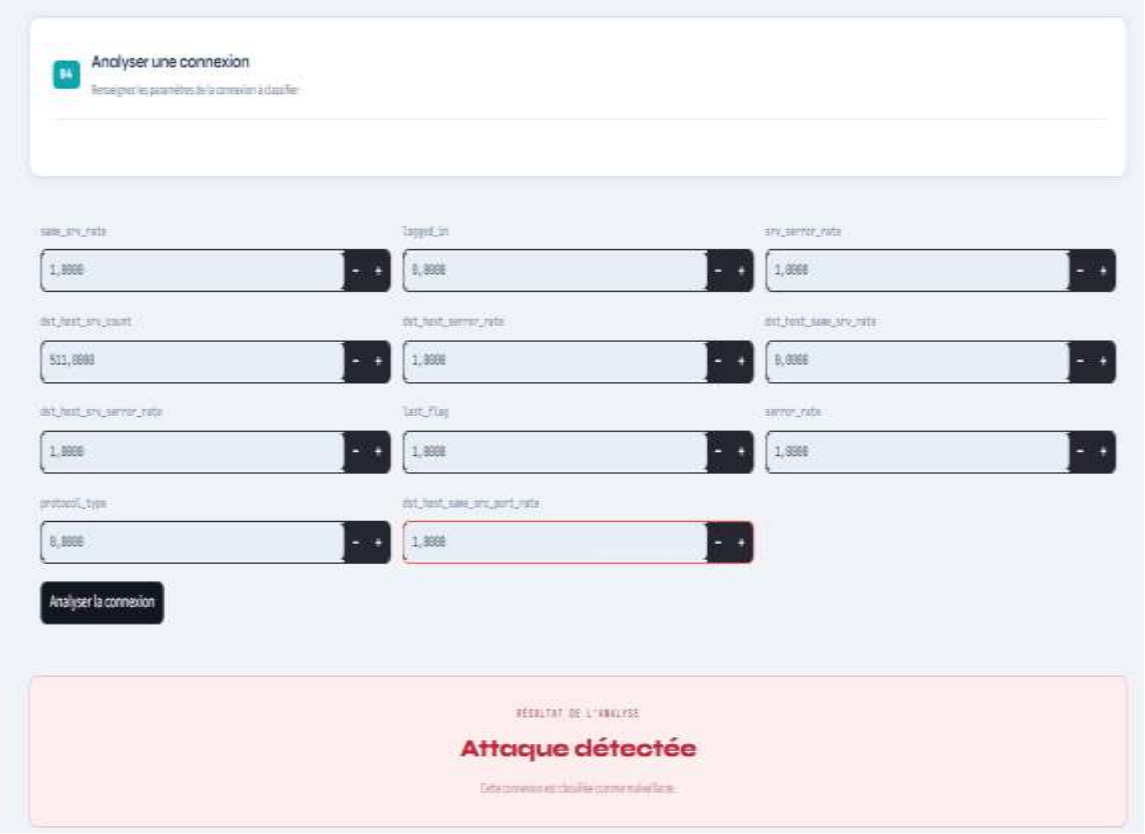
- Prétraitement automatique (encodage, normalisation, sélection des variables)



- Entraînement du modèle choisi et Évaluation avec accuracy et matrice de confusion



- Prédiction d'une nouvelle connexion réseau



L'utilisateur peut saisir les caractéristiques d'une connexion et obtenir directement une prédiction indiquant s'il s'agit d'un trafic normal ou d'une attaque.

CONCLUSION

Au terme de ce projet, nous avons pu concevoir et mettre en œuvre un système de détection d'intrusion réseau basé sur le Machine Learning, en suivant l'ensemble des étapes essentielles allant de la collecte et du prétraitement des données jusqu'à l'entraînement, l'évaluation et le déploiement d'un prototype interactif.

Ce travail nous a permis de mieux comprendre le rôle des systèmes de détection d'intrusion ainsi que l'intérêt du Machine Learning dans l'identification des comportements suspects au sein du trafic réseau.

Le système développé facilite la prise de décision en permettant de distinguer automatiquement le trafic normal des activités malveillantes, contribuant ainsi à renforcer la sécurité des systèmes informatiques.

Toutefois, la réalisation de ce projet a présenté certaines difficultés, notamment dans le traitement des données, le choix des modèles et l'interprétation des résultats, ainsi que dans l'intégration du modèle au sein d'une interface utilisateur.

Malgré cela, les résultats obtenus restent satisfaisants pour un projet académique et démontrent la pertinence de l'approche adoptée.

Enfin, plusieurs perspectives d'amélioration peuvent être envisagées, telles que l'utilisation de données plus récentes, l'intégration de techniques avancées comme le Deep Learning, l'optimisation des performances du modèle ou encore le déploiement dans un environnement réel avec un système d'alerte en temps réel, faisant de ce projet une base solide pour des travaux futurs en cybersécurité.

BIBLIOGRAPHIE

- Dataset utilisé : train.csv, test.csv
  https://www.kaggle.com/datasets/sampadab17/network-intrusion-detection
- Documentation Scikit-learn :
  https://scikit-learn.org
- Documentation officielle Python :
  https://www.python.org
- Documentation Streamlit :
  https://docs.streamlit.io
- Géron, A. (2019). HANDS-ON MACHINE LEARNING WITH SCIKIT-LEARN, KERAS & TENSORFLOW. O'Reilly